

CS-590 Database Design and Development

7-2 Assignment: Creating a Dashboard

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 16, 2026

Creating a Dashboard

Modern software systems rely heavily on data stored across multiple database platforms.

Organizations often use a combination of relational and non-relational databases to manage structured and unstructured data efficiently. The ability to combine, query, and visualize information from these databases allows analysts and developers to find patterns, check performance, and support data-driven decision making. Interactive dashboards are widely used tools for presenting data in a visual format that allows users to explore information quickly and intuitively. In this assignment, a Jupyter Notebook dashboard was created to visualize nutrient data stored within an existing database environment. The dashboard retrieves data from a MongoDB database and presents the information through interactive charts using the Dash and Plotly libraries. Python code within the notebook sets up the database connection, executes queries, processes the retrieved data, and generates interactive visualizations. This assignment shows how application-level tools such as Jupyter Notebook can be integrated with database systems to create interactive analytical dashboards.

Creating the Jupyter Notebook

To complete this assignment, a new Jupyter Notebook was created within the development environment. Jupyter Notebook provides an interactive platform that allows developers to combine Python code, data analysis, and visualizations within a single document. This

environment is particularly useful for building dashboards because it allows developers to execute database queries, process data, and at once visualize results within the same interface. The notebook serves as a vital part of the board system. It has all the Python scripts necessary to connect to the database, retrieve data, process the information into a usable format, and visualizations using the Dash framework. Each step of the process is organized into separate code cells to support clarity and readability. This structure allows the notebook to function both as a development environment and as documentation for how the dashboard was implemented.

Writing Python Code to Connect to the Database

Python code was written within the Jupyter Notebook to connect to the MongoDB database that stores the nutrient data. The PyMongo library was used to set up the connection between Python and the MongoDB database server. This library provides a simple interface that allows Python applications to interact with MongoDB collections and documents.

The connection process involves creating a MongoClient object that specifies the location of the MongoDB server. In this case, the database server is running locally on port 27017. After establishing the client connection, the code selects the proper database and collection that has the nutrient data.

Example connection code used in the notebook:

```
from pymongo import MongoClient
```

```
client = MongoClient("mongodb://localhost:27017/")
```

```
db = client["delisweet"]  
collection = db["nutrients"]
```

This code sets up the connection between the Jupyter Notebook and the MongoDB database. Once the connection is proved, Python can execute queries to retrieve data from the database. This integration shows how Python can function as a bridge between database systems and analytical applications.

Iterating on the Storyboard

Before developing the dashboard, a storyboard was created to outline the structure and functionality of the final visualization. The storyboard served as a planning tool that defined the layout of the dashboard, the types of charts that would be included, and the interactive components that users would be able to access.

During the implementation stage, minor adjustments were made to the storyboard to improve the usability and clarity of the dashboard. The original storyboard focused on displaying a single chart that visualized nutrient information. However, during development it became clear that added visualizations would offer better insight into the dataset. As a result, a second chart was added to show the distribution of nutrient measurement units.

Another adjustment involved adding an interactive filtering mechanism through a dropdown menu. This feature allows users to select specific nutrient units and dynamically update the charts. This improvement enhances the usability of the dashboard by allowing users to focus on specific portions of the dataset rather than viewing all data simultaneously.

These modifications improved the overall functionality of the dashboard while keeping the core structure outlined in the original storyboard.

Creating Queries to Populate the Dashboard

To populate the dashboard with data, Python queries were executed against the MongoDB nutrients collection. These queries retrieve the nutrient records stored in the database and convert them into a structured format that can be visualized.

The query used in the notebook retrieves all nutrient documents from the collection while excluding the MongoDB internal identifier field. This ensures that only relevant data fields are included in the dataset used by the dashboard.

Example query used in the notebook:

```
data = list(collection.find({}, {"_id":0}))
```

The returned data is then converted into a Pandas DataFrame. Pandas offer powerful data manipulation capabilities and allow the retrieved data to be organized into a tabular format. Once the data is stored in a DataFrame, it can easily be passed into visualization libraries such as Plotly to generate charts.

Executing the Queries

After the queries were created, they were executed within the Jupyter Notebook environment.

Executing the query retrieves the nutrient records stored within the MongoDB database and loads them into the notebook for processing.

The retrieved data is inspected to verify that the correct attributes have been returned. The notebook then performs basic preprocessing steps to ensure the data is suitable for visualization. These steps include removing duplicate records, handling missing values, and standardizing column names.

Once the data has been processed, the resulting dataset is stored in a DataFrame called `dashboard_df`. This DataFrame serves as the primary data source for the dashboard visualizations.

Creating the Interactive Dashboard

The dashboard was created using the Dash framework within the Jupyter Notebook environment.

Dash is a Python framework designed for building web-based data visualization applications.

The dashboard was implemented using interactive components such as dropdown menus and dynamic charts.

The dashboard holds two primary visualizations:

1. A bar chart that displays nutrient codes associated with each nutrient description.

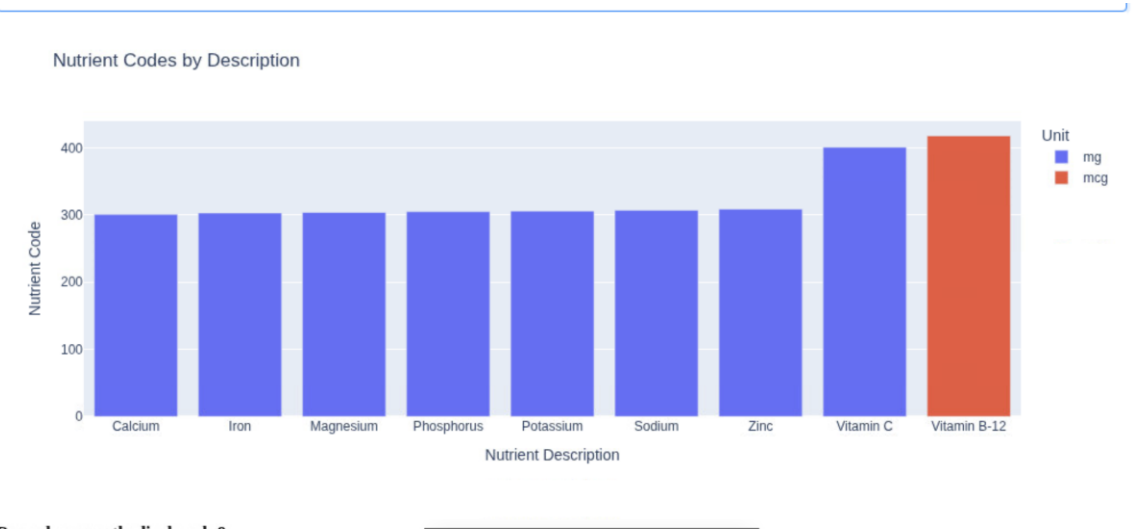
Delisweet Nutrient Dashboard

Use the dropdown to filter nutrient units.

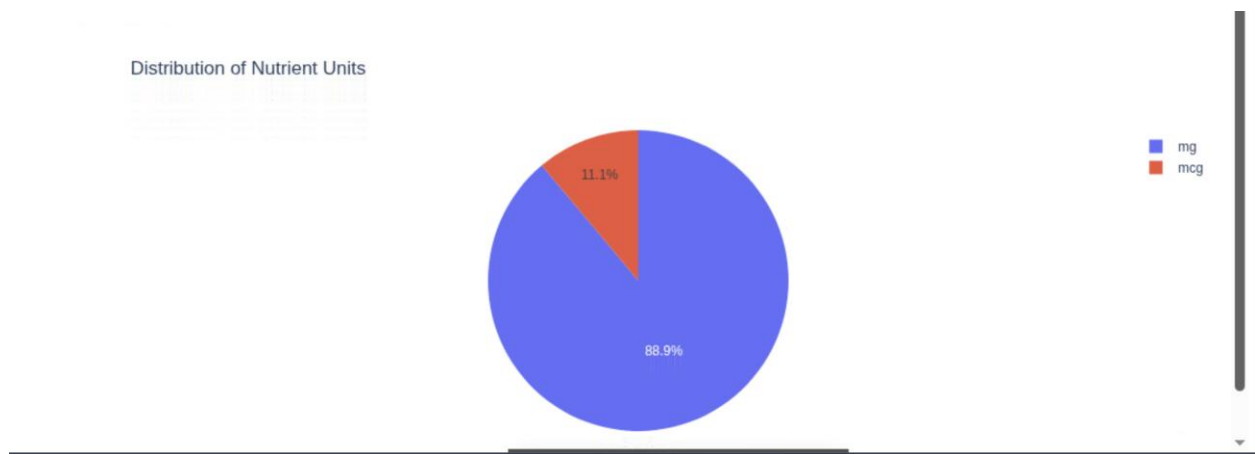
Select one or more nutrient units

mcg

mg



2. A pie chart illustrates the distribution of nutrient measurement units.



The layout of the dashboard includes a title, a dropdown filter, the primary chart, and a secondary chart that summarizes the data distribution. The dropdown part allows users to filter the dataset based on nutrient measurement units. When a user selects a value from the dropdown menu, the charts automatically update to reflect the filtered dataset.

Example code used to create the dashboard layout:


```
app.layout = html.Div([  
    html.H1("Delisweet Nutrient Dashboard"),  
    dcc.Dropdown(id="unit-filter", multi=True),  
    dcc.Graph(id="bar-chart"),  
    dcc.Graph(id="pie-chart")  
])
```

This interactive design allows users to explore the nutrient dataset dynamically rather than relying on static visualizations.

Dashboard Design Decisions

Several design decisions were made to ensure that the dashboard is still clear, informative, and easy to use. The primary goal of the dashboard was to provide a simple visualization of nutrient data stored within the MongoDB database.

The bar chart was selected as the main visualization because it allows users to quickly compare nutrient codes across different nutrient descriptions. This format makes it easy to show patterns and differences within the dataset.

The pie chart was included to provide an overview of how nutrient measurement units are distributed within the data. This visualization helps users quickly understand how the dataset is structured.

The dropdown filtering feature was implemented to allow users to interact with the dashboard. By selecting specific nutrient units, users can narrow the dataset and focus on certain nutrients. Interactive filtering improves the analytical value of the dashboard by allowing users to explore the data in greater detail.

Overall, the dashboard was designed to balance simplicity and functionality while ensuring that the visualizations stay easy to interpret.

Enhancing Dashboard Usability and Functionality

One potential improvement to the dashboard would be integrating data from multiple database systems. Currently, the dashboard retrieves information only from the MongoDB database. However, the assignment environment includes other databases such as PostgreSQL and Neo4j.

Enhancing the dashboard to combine data from multiple sources would significantly increase its analytical capabilities. For example, PostgreSQL could store structured food product data, MongoDB could store nutrient records, and Neo4j could be relationships between foods and nutrients within a graph structure.

Implementing this enhancement would require more Python code to connect to each database system using the right database drivers. Data retrieved from each database would then need to be transformed into a consistent format before being merged into a single dataset for visualization.

Added dashboard features could also include advanced filtering options, search functionality, and added chart types such as line graphs or heat maps. These improvements would make the dashboard more flexible and allow users to perform deeper analysis of the available data.

Python, Jupyter Notebook, and database technologies can be combined to create an interactive data visualization dashboard. By connecting the notebook to a MongoDB database, executing queries, and visualizing the results through Dash and Plotly, the dashboard offers a dynamic view of nutrient data stored within the database system. The implementation process involved creating database connections, retrieving and processing data, designing visualizations, and incorporating interactive elements such as filtering controls. The final dashboard allows users to explore the nutrient dataset through intuitive visual representations that support data analysis and decision making. Future improvements could expand the dashboard to incorporate multiple database systems and more visualization features. These enhancements would further show the power of database integration and interactive analytics in modern data-driven applications.